

Module 21

Pivoting & Lateral Movement

Routing through a controlled foothold: multi-homed hosts, SSH forwarding, SOCKS, ProxyJump, ProxyChains, route-based tunnels, segmentation, lateral-movement reasoning, telemetry, dan cleanup.

Learning path module ini

Network visibility -> multi-homed foothold -> local/dynamic/remote forwarding -> SOCKS -> internal enumeration -> ProxyJump lateral movement -> Ligolo-ng/Chisel mental model -> telemetry -> cleanup -> capstone.

Exit criteria

- Bisa membaca interface dan routing table lalu menjelaskan kenapa satu host dapat menjadi pivot tanpa mengaktifkan IP forwarding.
- Bisa membedakan SSH local (-L), dynamic (-D), remote (-R), dan ProxyJump (-J), termasuk arah listener dan connection origin.
- Bisa memakai SOCKS + ProxyChains secara terbatas pada TCP dan memahami kenapa beberapa scanner/protocol tidak cocok diproxy-kan.
- Bisa membedakan pivoting (network path) dari lateral movement (berpindah execution/authentication context ke host lain).
- Bisa memilih antara port-forward, SOCKS, TUN/routed tunnel, atau jump host berdasarkan kebutuhan, scope, dan blast radius.

Authorized-lab rule Hands-on module ini hanya untuk Docker lab lokal yang disediakan, GOAD/HTB/THM yang authorized, atau jaringan milik sendiri. Jangan membuat tunnel ke jaringan perusahaan/kampus/pihak lain tanpa Rules of Engagement yang eksplisit.

Field	Value
Learning path	Network visibility → multi-homed foothold → local/dynamic/remote forwarding → SOCKS → internal enumeration → ProxyJump lateral movement → Ligolo-ng/Chisel mental model → telemetry → cleanup → capstone
Safety model	Docker lab lokal terisolasi + dummy credential. Internal service tidak dipublish ke host; attacker container hanya terhubung ke external lab network, jump host terhubung ke dua network. Tidak ada password spraying, credential dumping, hash replay, ticket abuse, atau remote-service abuse
Training credentials	jump: pivot/Pivot21! · internal SSH: student/Student21! — disposable lab only, jangan reuse di sistem lain

Exit criteria

- [] Bisa membaca interface dan routing table lalu menjelaskan kenapa satu host bisa menjadi pivot **tanpa** mengaktifkan IP forwarding.
- [] Bisa membedakan SSH local (-L), dynamic (-D), remote (-R), dan ProxyJump (-J), termasuk arah listener dan connection origin.
- [] Bisa memakai SOCKS + ProxyChains secara terbatas pada TCP dan memahami kenapa beberapa scanner/protocol tidak cocok diproxy-kan.
- [] Bisa membedakan pivoting (network path) dari lateral movement (berpindah execution/authentication context ke host lain).
- [] Bisa memilih antara port-forward, SOCKS, TUN/routed tunnel, atau jump host berdasarkan kebutuhan, scope, dan blast radius.

1. Mental Model: Reachability Bukan Ownership

Pivoting memperluas jalur network dari posisi yang sudah kita miliki. **Lateral movement** adalah penggunaan identity/session/remote-access path untuk memperoleh execution context di host lain. Keduanya sering berurutan tapi **bukan hal yang sama**.

Concept	Pertanyaan inti	Contoh
Segmentation	siapa bisa berbicara langsung ke siapa?	attacker tidak punya route ke subnet internal
Pivot	host mana punya dua visibility zone?	jump host punya interface external + internal
Tunnel	bagaimana traffic melewati pivot?	SSH -L/-D/-R, Chisel, Ligolo-ng
Lateral movement	identity/session pindah ke host mana?	SSH via ProxyJump ke internal host
Objective	akses minimum apa yang diperlukan?	reach internal web, bukan scan seluruh subnet

Topologi lab

```
ATTACKER (172.31.210.10)
| external only
v
JUMP (172.31.210.20 + 172.31.211.20)
| internal visibility
+-----> WEB 172.31.211.30:80
+-----> SSH 172.31.211.40:22
```

Lab 01 — Workspace & Topology

```
cd modules/21-pivoting-lateral-movement/labs
./workspace-init.sh module21-pivoting
python topology-report.py
```

topology-report.py membaca topology.json (4 node: attacker/operator, jump/pivot, internal-web/internal service, internal-ssh/lateral target) dan mencetak role + interface tiap node.

2. Routing, Interfaces, dan Multi-Homed Hosts

Sebelum membuat tunnel, identifikasi interface, route, dan reachable network. Host multi-homed bisa mengakses lebih dari satu segment, tapi itu **tidak otomatis berarti kernel melakukan forwarding** antar-segment. SSH forwarding bekerja karena proses sshd pada jump host membuka koneksi **baru** ke destination dari network namespace miliknya sendiri.

Command	Jawaban yang dicari
ip -brief addr	interface + address apa yang ada?
ip route	network mana yang directly connected/lewat gateway?
ip route get <IP>	kernel memilih path/interface mana?
ss -lntp	listener apa yang aktif dan di address mana?
curl/nc	apakah protocol benar-benar reachable?

```
omarchy pkg add openssh proxychains-ng nmap curl netcat-openbsd
# iproute2 biasanya sudah bagian dari base system; cek: ip -V
```

Lab 02 — Build Isolated Pivot Lab

```
./pivot-lab.sh build
./pivot-lab.sh create
./pivot-lab.sh status
./segmentation-check.sh
```

Topologi Docker-nya: rt21_external (172.31.210.0/24, bridge biasa) dan rt21_internal (172.31.211.0/24, bridge -- internal — tidak ada default route keluar). Attacker hanya join external; jump join **keduanya**; web & internal-ssh hanya join internal.

Expected result.

Dari attacker container, 172.31.210.20:22 reachable. 172.31.211.30 **tidak** dapat diakses langsung. Ini membuktikan segmentation sebelum kita membuat tunnel apa pun.

3. SSH Local Port Forward (-L)

Local forwarding membuat listener di sisi **client**. Setiap koneksi ke listener lokal diteruskan lewat SSH channel, lalu sshd di jump host membuat koneksi ke destination yang kita tentukan.

```
local client          jump          internal
127.0.0.1:8088 --SSH channel--> 172.31.210.20 --TCP connect--> 172.31.211.30:80
```

Lab 03 — Forward Satu Internal Web Service

```
# terminal A: masuk attacker container
./pivot-lab.sh shell

ssh -N \
-o ExitOnForwardFailure=yes \
-L 127.0.0.1:8088:172.31.211.30:80 \
pivot@172.31.210.20
# password training: Pivot21!
```

```
# terminal B: masuk attacker container lagi
curl -i http://127.0.0.1:8088/
```

- Bind ke 127.0.0.1 supaya listener tidak terekspose ke container/network lain.
- Gunakan -N kalau tunnel tidak butuh remote shell.
- ExitOnForwardFailure=yes membuat command gagal kalau listener tidak bisa dibangun — bukan diam-diam lanjut.

Think in direction.

-L cocok saat operator ingin mengekspos **satu** destination internal sebagai port lokal. Ini paling kecil blast radius-nya dibanding membuat seluruh subnet routable.

4. Dynamic Forwarding (-D) + SOCKS + ProxyChains

Dynamic forwarding membuat SSH client bertindak sebagai SOCKS4/5 server. Aplikasi yang memahami SOCKS bisa meminta koneksi ke banyak destination lewat **satu** SSH channel. ProxyChains memaksa program TCP yang tidak punya native proxy support melewati SOCKS/HTTP proxy.

Lab 04 — Buat SOCKS Listener

```
# terminal A di attacker container
ssh -N \
-o ExitOnForwardFailure=yes \
-D 127.0.0.1:1080 \
pivot@172.31.210.20
```

Lab 05 — Akses Internal Web via ProxyChains

```
# terminal B di attacker container
proxychains4 -f /etc/proxychains4.conf \
curl -s http://172.31.211.30/
```

Lab 06 — Internal Enumeration yang Compatible dengan SOCKS

```
proxychains4 -f /etc/proxychains4.conf \
nmap -sT -Pn -n -p 22,80 \
172.31.211.30 172.31.211.40
```

ProxyChains limitation.

ProxyChains-NG me-redirect koneksi TCP dari program *dynamically linked*. Ia **tidak** membawa ICMP/UDP, dan pcap/raw-packet scan tidak cocok. Untuk Nmap gunakan TCP connect (-ST), numeric IP, port terbatas, dan validasi manual.

5. ProxyJump (-J) dan Lateral Movement

ProxyJump meminta OpenSSH membuat koneksi ke destination **melalui** jump host. Ini bukan "scan through proxy" — ini transport SSH ke host akhir. Dalam lab, credential target sudah disediakan dan **tidak** diperoleh lewat spraying atau dumping.

Lab 07 — SSH ke Internal Host Melalui Jump

```
# dari attacker container
ssh -J pivot@172.31.210.20 \
student@172.31.211.40
# pivot -> Pivot21!
# student -> Student21!

# setelah login ke internal host
hostname
id
ip -brief addr
ip route
ss -lnt
```

Lateral-movement boundary.

Di titik ini terjadi perubahan execution context: command sekarang berjalan pada `internal-ssh` sebagai user `student`. Itu berbeda dari sekadar mengirim traffic lewat jump (§3–4) — di sini *identitas* dan *tempat eksekusi* yang berpindah.

Safer SSH config pattern

```
Host rt21-jump
  HostName 172.31.210.20
  User pivot

Host rt21-internal
  HostName 172.31.211.40
  User student
  ProxyJump rt21-jump
```

6. SSH Remote Forward (-R): Membalik Arah Listener

Remote forwarding membuat listener pada sisi **SSH server**. Koneksi yang masuk ke listener remote dibawa lewat SSH channel dan diteruskan ke destination di sisi client. Ini sering membingungkan karena arah listener **berlawanan** dengan -L.

Lab 08 — Remote Listener Hanya di Localhost Jump

```
# terminal A di attacker container
python3 -m http.server 9000 --bind 127.0.0.1

# terminal B di attacker container
ssh -N \
  -o ExitOnForwardFailure=yes \
  -R 127.0.0.1:9001:127.0.0.1:9000 \
  pivot@172.31.210.20

# terminal C di Omarchy host
./pivot-lab.sh jump-shell
curl -I http://127.0.0.1:9001/
```

Security detail.

Bind address eksplisit + `GatewayPorts=no` mencegah remote-forward listener tersedia ke seluruh network jump. Dalam engagement nyata, listener exposure harus selalu masuk design review.

7. Choosing the Tunnel Primitive

Need	Good starting primitive	Trade-off
1 internal TCP service	SSH -L	simple, minimal exposure, one mapping per service
many TCP destinations	SSH -D + SOCKS	flexible; app/protocol compatibility matters
SSH login through bastion	ProxyJump	clean native SSH workflow; SSH-specific
remote-side listener back to operator	SSH -R	direction gampang salah konfigurasi; bind carefully
route-like access to subnet	Ligolo-ng TUN	natural routing + TCP/UDP/ICMP; TUN/route changes on proxy
HTTP-carried forward/reverse tunnel	Chisel	single binary, SSH-secured-over-HTTP; tetap proses + long-lived channel

Lab 09 — Record Why, Not Just How

```
python activity-log.py module21-pivoting/06-evidence/activity.tsv \
  --action 'dynamic-socks' \
  --source attacker \
  --destination '172.31.211.0/24 via jump' \
  --purpose 'enumerate only TCP 22/80 in training lab'
```

8. Ligolo-ng: Route-Based Pivoting Mental Model

Ligolo-ng memakai TUN interface di sisi proxy/operator dan userland network stack di sisi agent. Hasilnya, operator menambahkan **route** ke subnet agent dan banyak tool bisa memakai normal socket/routing **tanpa** ProxyChains. Agent side tidak selalu butuh privilege tinggi; proxy side perlu kemampuan membuat TUN interface.

```

Operator route table
172.31.211.0/24 -> ligolo TUN
      |
      v
Ligolo proxy === encrypted connection === Ligolo agent
      |
      +--> internal network

```

Lab 10 — Route Planning (Tanpa Deploy Agent)

- Dari `topology.json`, identifikasi network yang akan ditambahkan sebagai route: `172.31.211.0/24`.
- Tulis evidence yang harus diambil **sebelum** route ditambah: `ip route`, interface list, scope, expected next hop.
- Tulis cleanup: stop tunnel, hapus route/interface yang dibuat, verifikasi route table kembali ke baseline.

Certificate hygiene.

Ligolo-ng mendukung self-signed TLS + fingerprint verification. Opsi yang mengabaikan certificate verification sebaiknya hanya dipakai debugging lab — jangan jadikan default operasi.

9. Chisel: HTTP Transport + SSH Security

Chisel adalah single-binary TCP/UDP tunnel yang dibawa lewat HTTP dan diamankan dengan SSH — mendukung normal forward, reverse forward, SOCKS, authentication, server fingerprint, dan beberapa tunnel dalam satu connection. Berguna ketika SSH forwarding native tidak tersedia tapi HTTP connectivity ada.

Lab 11 — Tool Provenance & Help-only Inspection

```

# optional; tidak diperlukan untuk core lab
docker run --rm jpillora/chisel --help

# alternatif: build dari source pada pinned/reviewed version
# daripada menjalankan curl | shell di workstation security-sensitive

```

- Sebelum deploy: catat version/source, authentication mode, fingerprint/TLS model, bind address, allowed remotes, dan cleanup plan.
- Reverse forwarding memperluas listener surface — jangan pakai wildcard bind kalau localhost sudah cukup.
- Perubahan upstream bisa memengaruhi ACL/SOCKS behavior; baca README/release notes versi yang benar-benar dipakai.

10. Multi-Hop Pivot Chains: Complexity Naik Cepat

Satu jump host masih mudah dipahami. Dua atau tiga pivot membuat routing, DNS, credential scope, failure domain, dan cleanup jauh lebih kompleks. **Jangan membuat chain hanya karena tool mendukungnya** — pilih chain minimum yang dibutuhkan untuk objective.

```

Operator -> Jump A -> Jump B -> Internal Service

Setiap hop menambah:
+ authentication event
+ long-lived connection
+ failure point
+ listener/route state
+ cleanup obligation

```

Lab 12 — Multi-hop Design Tabletop

- Tanpa menambah container baru, gambar hypothetical second jump di `172.31.211.50` yang punya akses ke `172.31.212.0/24`.
- Tentukan apakah objective hanya satu TCP service (pilih chained local forward/ProxyJump) atau butuh route-like access (pertimbangkan TUN-based tunnel).
- Tuliskan failure behavior kalau Jump A restart, Jump B kehilangan route, atau satu credential dicabut.

Operator rule.

Setiap hop harus punya owner, purpose, start time, expected end time, bind/route state, dan cleanup record. Chain tanpa inventory hampir selalu menghasilkan orphan tunnel.

11. DNS Through a Pivot: Sering Jadi Sumber False Negative

Internal hostname sering hanya dikenal oleh resolver internal. Kalau aplikasi resolve nama **sebelum** traffic masuk tunnel, query bisa bocor ke resolver operator atau gagal total.

Pattern	Resolution location	Risk/note
Numeric IP	none	paling deterministik untuk lab/known assets
Local DNS	operator resolver	internal zone mungkin tidak dikenal; bisa leak query

ProxyChains proxy_dns	melalui proxy mechanism	hanya cocok untuk tool yang benar-benar ter-hook
Route-based + internal DNS	operator mengakses resolver via route	lebih natural, tapi scope resolver juga harus authorized

Lab 13 — DNS Decision Record

- Untuk core Docker lab gunakan numeric IP supaya hasil tidak bergantung DNS.
- Tuliskan bagaimana strategi berubah kalau target diberi nama `app.internal.example` dan hanya resolver `172.31.211.53` yang mengetahuinya.
- Catat DNS sebagai bagian dari tunnel plan — bukan troubleshooting terakhir.

12. Lateral Movement: Transport, Identity, dan Execution Context

Lateral movement harus dibaca sebagai kombinasi **transport + authentication material + authorization + remote execution/session primitive**. Module ini hanya mempraktikkan SSH dengan training credentials; protocol lain dibahas sebagai decision model saja — supaya tidak tercampur dengan credential abuse.

Method	Needs	Execution/session result	Telemetry examples
SSH	network + valid account/key	shell/command session	sshd auth, process tree, source IP
RDP	network + interactive-logon right	desktop session	4624/logon type, RDP operational logs
WinRM	network + authorized account/policy	PowerShell/remoting session	WinRM + PowerShell logs
SMB/RPC remote admin	network + appropriate admin rights	service/task/RPC dependent	SMB/RPC + service/task events

No credential guessing here.

Password spraying, hash replay, ticket abuse, credential dumping, dan remote-service abuse **tidak** dilakukan di module ini. Kalau identity belum authorized untuk target, berhenti pada path hypothesis.

Lab 14 — Lateral Evidence Record

```
python activity-log.py module21-pivoting/06-evidence/activity.tsv \
--action 'proxyjump-login' \
--source 'attacker via jump' \
--destination '172.31.211.40:ssh' \
--purpose 'authorized student shell for topology validation'
```

13. Tunnel Security & Credential Hygiene

- Jangan aktifkan SSH agent forwarding (-A) secara default — host remote yang bisa mengakses forwarded agent socket dapat memakai agent itu untuk autentikasi selama koneksi hidup.
- Bind listener ke `127.0.0.1` kecuali RoE memang membutuhkan listener di interface lain.
- Gunakan host-key/fingerprint verification. Convenience option yang mematikan certificate/host verification hanya untuk debugging disposable lab.
- Jangan menaruh password/token di command line, process list, shell history, atau README repo. Training password di module ini sengaja disposable.
- Gunakan `ServerAliveInterval/ServerAliveCountMax` untuk menghindari tunnel zombie, dan `ExitOnForwardFailure` untuk fail-fast.

```
ssh -N -o ExitOnForwardFailure=yes -o ServerAliveInterval=30 -o ServerAliveCountMax=3 \
-L 127.0.0.1:8088:172.31.211.30:80 pivot@172.31.210.20
```

Lab 15 — Inspect Tunnel State

```
# attacker container
ss -lntp | grep -E ':(8088|1080|9000)' || true
ps aux | grep '[s]sh .*172.31.210.20'

# jump container from Omarchy host
./pivot-lab.sh jump-shell
ss -tnp
ps aux | grep '[s]shd'
```

14. Segmentation, DNS, dan Tool Behavior Setelah Pivot

Setelah tunnel bekerja, **jangan langsung melakukan full-subnet scan**. Bangun inventory bertahap: route → known hosts → known ports → protocol verification.

Problem	Symptom	Response
No route	network unreachable/timeout	verify <code>ip route</code> + intended tunnel state
SOCKS works, ping fails	expected: ICMP not proxied	gunakan TCP validation atau route-based tunnel
Name fails but IP works	DNS not traversing intended path	gunakan numeric IP atau controlled remote DNS
Nmap strange through SOCKS	raw/pcap method incompatible	gunakan <code>-ST</code> , small port set, manual verify
Forward listener unreachable	wrong bind or collision	<code>ss -lntp</code> + explicit <code>127.0.0.1</code> bind

Lab 16 — Evidence & Cleanup

```
# capture before cleanup
ip route > module21-pivoting/06-evidence/attacker-routes.txt
ss -lntp > module21-pivoting/06-evidence/listeners.txt
./evidence-manifest.sh

# stop SSH forward processes, exit lab shells, then:
./pivot-lab.sh cleanup
```

15. Detection-Aware View

Pivoting menghasilkan telemetry meskipun payload tidak dieksekusi pada internal target. Defender bisa melihat long-lived SSH/TLS/HTTP connections, new listener, process execution, authentication, unusual east-west connections, serta workstation yang tiba-tiba jadi connection broker.

Activity	Potential telemetry
SSH - L/-D/-R	sshd auth/session logs, long-lived connection, local/remote listener, child connections
ProxyJump lateral login	authentication logs di jump + target, new user session/processes
ProxyChains enumeration	banyak TCP connect bersumber dari pivot/jump context
Ligolo-ng/Chisel	new binary/process, persistent encrypted egress, route/TUN/listener changes
Cleanup failure	orphan listener/process/route tersisa setelah engagement

Blue Team perspective.

Network segmentation bukan hanya firewall rule. Monitor siapa yang menjadi *unexpected bridge* antar zone: dual-homed host, admin workstation, jump server, VPN endpoint, CI runner, dan management interface.

16. Capstone — Reach the Internal Zone with Minimum Exposure

Scenario: attacker hanya berada di external Docker network. Objective: buktikan internal web reachable dan lakukan **satu** authorized SSH login ke internal host lewat jump, dengan exposure minimum dan evidence yang cukup untuk direproduksi.

- **Phase A — Baseline:** buktikan direct access ke `172.31.211.30` gagal dari attacker.
- **Phase B — Local forward:** expose hanya internal web sebagai `127.0.0.1:8088` dan capture HTTP evidence.
- **Phase C — Dynamic SOCKS:** enumerate hanya TCP 22 dan 80 pada dua known internal host; jangan scan /24 penuh.
- **Phase D — Lateral movement:** gunakan provided training credentials + ProxyJump untuk login ke `172.31.211.40` sebagai `student`.
- **Phase E — Evidence:** simpan route/listener/output, timestamp activity, hash evidence.
- **Phase F — Cleanup:** stop tunnel, exit session, remove container/network, verifikasi tidak ada listener training tersisa.

Deliverable

Artifact	Minimum content
----------	-----------------

topology.md	zones, node IPs, kenapa jump adalah pivot candidate
tunnel-plan.md	chosen primitive + bind + destination + reason
activity.tsv	timestamped actions and intended scope
evidence/	curl/nmap/ssh observations + route/listener snapshots
cleanup.md	process/listener/network removed + validation commands

17. Cheat Sheet & Common Mistakes

Need	Pattern
Local forward	ssh -N -L 127.0.0.1:LPORT:DEST:DPORT user@jump
Dynamic SOCKS	ssh -N -D 127.0.0.1:1080 user@jump
Remote forward	ssh -N -R 127.0.0.1:RPORT:LOCAL:LPORT user@jump
SSH bastion	ssh -J user@jump user@internal
Route check	ip -brief addr ; ip route ; ip route get DEST
Listener check	ss -lntp
SOCKS TCP scan	proxychains4 nmap -sT -Pn -n -p <small-set> <known-host>

Common mistake.

Mengikat listener ke 0.0.0.0 ketika 127.0.0.1 cukup, menjalankan full subnet scan begitu tunnel hidup, menganggap SOCKS membawa semua protocol, lupa DNS path, atau meninggalkan tunnel/process setelah cleanup.

18. Knowledge Check

- Kenapa dual-homed host tidak otomatis L3-forward?
- Listener -L, -D, dan -R berada di sisi mana?
- Kenapa ProxyChains + Nmap harus pilih -sT?
- Pivoting beda apa dengan lateral movement?
- Kapan single forward lebih baik dari routed tunnel?
- Kenapa bind address + DNS path masuk threat model?
- Telemetry apa yang muncul saat host menjadi pivot?
- Bukti minimum cleanup tunnel berhasil apa saja?

19. Deliverables

```
21-pivoting-lateral-movement/
├── README.md
├── docx/
├── pdf/
├── labs/
│   ├── workspace-init.sh      - scaffold workspace (00-scope ... 06-evidence)
│   ├── topology-report.py    - cetak role/interface tiap node dari topology.json
│   ├── topology.json         - definisi topologi: attacker/jump/internal-web/internal-ssh
│   ├── pivot-lab.sh          - build/create/status/shell/jump-shell/cleanup Docker lab
│   ├── segmentation-check.sh - verifikasi segmentation sebelum tunnel dibuat
│   ├── activity-log.py        - append TSV: action/source/destination/purpose
│   ├── evidence-manifest.sh  - SHA-256 manifest workspace
│   └── pivot-lab/
│       ├── attacker/ (Dockerfile + proxychains.conf)
│       ├── jump/ (Dockerfile + sshd_config, dual-homed)
│       ├── web/ (Dockerfile + index.html, internal-only)
│       └── internal-ssh/ (Dockerfile + sshd_config, lateral target)
```

Topologi Docker: rt21_external (172.31.210.0/24) dan rt21_internal (172.31.211.0/24, bridge --internal). Attacker hanya di external; jump di keduanya; web & internal-ssh hanya di internal.

20. References

- OpenSSH `ssh(1)` – forwarding + ProxyJump – <https://man.openbsd.org/ssh.1>
- Linux `Ssh(1)` – forwarding semantics – <https://man7.org/linux/man-pages/man1/ssh.1.html>
- `ip-route(8)` – routing table management – <https://man7.org/linux/man-pages/man8/ip-route.8.html>
- ProxyChains-NG – SOCKS/TCP limitations – <https://github.com/rofl0r/proxychains-ng>
- Arch Linux – proxychains-ng/OpenSSH – https://archlinux.org/packages/extra/x86_64/proxychains-ng/
- Ligolo-ng docs – TUN/routing model – <https://docs.ligolo.ng/>
- Chisel official repository – <https://github.com/jpillora/chisel>
- Docker networking overview – <https://docs.docker.com/engine/network/>

Next module:

22-threat-intelligence-adversary-emulation – intelligence-driven hypotheses dan authorized adversary emulation planning.